Programming Fundamentals

Week 7 — Monday: Persistence, Files and Input Validation

Programming Fundamentals Team

SETU

2026-10-28

Today: Trust the Input, Keep the Data

By the end of the class you should be able to:

- explain why programs need **persistence**
- save and retrieve data using text files and JSON
- validate input using `if` statements and loops
- combine both ideas in a small application

Our Dog Day Care Problem

```
dogs = [{"name": "Buddy", "age": 4}]
dogs.append({"name": "Luna", "age": 2})
```

Two questions:

- What happens to Luna when we close the program?
- What happens if someone enters an age of `-3`?

Our Dog Day Care Problem

```
dogs = [{"name": "Buddy", "age": 4}]
dogs.append({"name": "Luna", "age": 2})
```

Two questions:

- What happens to Luna when we close the program?
- What happens if someone enters an age of `-3`?

We need **persistence** and **validation**.

Persistence: Data That Survives

Persistence means keeping information after the program stops.

- A Python list exists in memory while the program runs.
- A file stores information so we can retrieve it later.
- A typical program **loads** saved data, changes it, then **saves** it.

Write to a Text File

```
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("First note\n")
```

- `open()` opens a file.
- `"w"` creates or **overwrites** it.
- `with` closes it automatically.

Predict: What happens if we run the code twice?

Why Use `with open()`?

```
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("Remember to save!\n")
```

- `with` manages the file for us.
- The file is closed automatically when the block finishes.
- The **indented block** shows where the file is being used.

Why is closing a file important?

Three Useful File Modes

- `"w"` — **write**: create or overwrite a file
- `"a"` — **append**: add to the end
- `"r"` — **read**: open an existing file

Warning: `"w"` replaces the old contents. It does not add to them.

Append Without Erasing

```
with open("notes.txt", "a", encoding="utf-8") as file:
    file.write("Another note\n")
```

`"a"` adds to the end of the file.

Append Without Erasing

```
with open("notes.txt", "a", encoding="utf-8") as file:
    file.write("Another note\n")
```

`"a"` adds to the end of the file.

Quick check: Which mode would you use for a running journal: `"w"` or `"a"`?

Read Back What We Saved

```
with open("notes.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line.strip())
```

- `"r"` reads an existing file.
- The loop processes one line at a time.
- `strip()` removes the newline when displaying it.

The file must exist. File-not-found errors and exception handling are covered in Week 8.

What If the File Does Not Exist?

```
with open("missing.txt", "r", encoding="utf-8") as file:
    print(file.read())
```

If `missing.txt` does not exist, Python raises `FileNotFoundError`.

- Reading (`"r"`) needs an existing file.
- Writing (`"w"`) or appending (`"a"`) can create one.

Activity 1 — Make a Journal

Quick code along

1. Ask for one note and append it to `journal.txt`.
2. Read the file and display the notes.
3. Run the program twice: do earlier notes remain?

Journal — Let's Write the Code

Code along - Solution

```
note = input("Enter a journal note: ")

with open("journal.txt", "a", encoding="utf-8") as file:
    file.write(note + "\n")

print("\nYour journal:")
with open("journal.txt", "r", encoding="utf-8") as file:
    print(file.read())
```

Predict: What does `"a"` do? Why do we add `"\\n"`?

Journal — Run It Twice!

First run:

```
Enter a journal note: Learned about files

Your journal:
Learned about files
```

Second run:

```
Enter a journal note: Practised Python

Your journal:
Learned about files
Practised Python
```

Why Plain Text Is Not Always Enough

Our dog records contain **several related values**:

```
dogs = [
    {"name": "Buddy", "age": 4},
    {"name": "Luna", "age": 2}
]
```

How can we save the **list of dictionaries** and load it again?

Why Plain Text Is Not Always Enough

Our dog records contain **several related values**:

```
dogs = [
    {"name": "Buddy", "age": 4},
    {"name": "Luna", "age": 2}
]
```

How can we save the **list of dictionaries** and load it again?

Use **JSON** — a text format for structured data.

Saving Structured Data with JSON

```
import json

with open("dogs.json", "w", encoding="utf-8") as file:
    json.dump(dogs, file, indent=4)
```

`json.dump()` writes our list of dictionaries to a file.

`indent=4` makes the saved file readable.

Loading Structured Data with JSON

```
import json

with open("dogs.json", "r", encoding="utf-8") as file:
    dogs = json.load(file)

print(dogs[0]["name"])
```

`json.load()` gives us a Python list or dictionary again.

The Load → Change → Save Pattern

```
START PROGRAM
    LOAD saved records
    DISPLAY menu
    GET user input
    CHANGE records in memory
    SAVE updated records
END PROGRAM
```

Why must we save after **making a change?**

Input Is Not Always Valid

```
age = int(input("Enter the dog's age: "))
```

What if the user enters:

- `-4`?
- `1000`?
- `hello`?

The first two can be rejected with **conditions**.

> Non-numeric input raises ValueError. We'll learn how to handle that with try/except in Week 8.

Validation: Check Before Accepting

Input validation checks whether a value meets our rules.

```
age = int(input("Age (0 to 25): "))

if 0 <= age <= 25:
    print("Age accepted")
else:
    print("Age must be between 0 and 25")
```

Question: Does this let the user try again?

Keep Asking Until It Is Valid

```
age = int(input("Age (0 to 25): "))

while age < 0 or age > 25:
    print("Please enter an age from 0 to 25")
    age = int(input("Age (0 to 25): "))

print("Accepted:", age)
```

The loop repeats while the age is **outside** our range.

This assumes numeric input. Handling letters is a separate Week 8 topic.

Prediction: Will the Loop Run?

```
while age < 0 or age > 25:
    age = int(input("Try again: "))
```

What happens when `age` is:

- `-1`?
- `12`?
- `26`?

Prediction: Will the Loop Run?

```
while age < 0 or age > 25:
    age = int(input("Try again: "))
```

What happens when `age` is:

- `-1`?
- `12`?
- `26`?

Discuss: Why is `or` correct here, rather than `and`?

Validate a Menu Choice

```
choice = input("Choose A, V or Q: ").strip().upper()

while choice not in ["A", "V", "Q"]:
    print("Invalid choice")
    choice = input("Choose A, V or Q: ").strip().upper()
```

`strip()` removes extra spaces.

`upper()` allows lowercase input.

Put Validation in a Function

```
def get_menu_choice():
    choice = input("A=Add, V=View, Q=Quit: ").strip().upper()
    while choice not in ["A", "V", "Q"]:
        print("Please enter A, V or Q")
        choice = input("A=Add, V=View, Q=Quit: ").strip().upper()
    return choice
```

Why is a function better than repeating this code throughout a program?

Validate Before Saving

```
name = input("Dog's name: ").strip()
while name == "":
    name = input("Name cannot be blank. Try again: ").strip()

age = int(input("Age (0 to 25): "))
while age < 0 or age > 25:
    age = int(input("Enter an age from 0 to 25: "))

dogs.append({"name": name, "age": age})
```

Key idea: Check data **before** adding it to the list or saving the file.

Check Your Understanding

Quick questions:

- Which mode would you use to add a line without erasing existing data?
- What does `with` help us do?
- What is the difference between `json.dump()` and `json.load()`?
- What happens if you read a file that does not exist?

Before Wednesday

Today: We made data survive and checked it before saving.

Wednesday: We'll use JSON to store the **PC Troubleshooter's** questions and rules outside the Python program.

Next week: Debugging and exception handling — including what to do when a user enters letters instead of a number, or a file cannot be opened.